COMPUTER SCIENCE + COMPUTER ENGINEERING

Laboratory and Project Workbook

Practice / evidence / integration

Original laboratory briefs with concrete implementation steps, written exercises and project reviews.

20 practical briefs · 50 problems · 25 review questions

Package 05 · Expanded Volume 04 · September 28, 2026
Self-study teaching edition. Completion requires independent implementation, testing and review.

Contents

Module 16: Processor architecture and assembly 4

Lab 16.1: Instruction sets, registers and calling conventions 4

Lab 16.2: Pipelines, dependencies and branch prediction 5

Lab 16.3: Caches, locality and memory hierarchy 6

Lab 16.4: Virtual memory, SIMD and architectural limits 7

Module 17: Operating systems and resource management 9

Lab 17.1: Processes, threads, system calls and protection 9

Lab 17.2: Allocation, paging and memory-mapped data 10

Lab 17.3: Scheduling, responsiveness and real-time constraints 11

Lab 17.4: Filesystems, persistence and asynchronous I/O 12

Module 18: Concurrency and parallel programming 14

Lab 18.1: Races, mutexes and happens-before 14

Lab 18.2: Condition variables, deadlocks and cancellation 15

Lab 18.3: Atomics, memory ordering and false sharing 16

Lab 18.4: Task graphs, work stealing and scaling limits 17

Module 19: Networks, storage and data services 19

Lab 19.1: Network layers, latency and transport semantics 19

Lab 19.2: Protocol design, serialization and timeouts 20

Lab 19.3: Relational modeling, SQL and indexes 21

Lab 19.4: Transactions, consistency and distributed failure 22

Module 20: Engineering practice, security and maintainability 24

Lab 20.1: Requirements, architecture and design records 24

Lab 20.2: Defensive programming and threat modeling 25

Lab 20.3: Build automation, dependencies and release discipline 26

Lab 20.4: Ethics, accessibility, teamwork and sustainable delivery 27

Package 05 · Expanded Volume 04

Work from the original brief to the concrete steps, then challenge the implementation with new cases. Do not replace a systems investigation with a passing helper-function test. Record source revision, toolchain, operating system, chosen limits, exact commands and evidence location for every submission.

This workbook contains 20 practical briefs, 50 additional problems, 25 module-review questions, 12 readiness questions and five integration-project checklists. The Gate D project overlaps Module 20; it is not a sixth unrelated project. Sixteen labs have coding interfaces; the four engineering-practice labs require written/review/build evidence.

Readiness diagnostic

Answer unaided. These questions identify prerequisite repairs, not a credential. Explain the reason, not just the term. With a weak answer on lifetime, ownership, arithmetic or graph ordering, revisit Modules 11–15 before using that concept in concurrent code.

1. **What is wrong with returning a reference to a function-local object?**
2. **Does vector.reserve(100) create 100 live elements?**
3. **Why is ordinary signed integer overflow not a suitable wrapping model?**
4. **Who owns the target of a raw pointer?**
5. **What must an exception-safe replacement preserve when validation fails?**
6. **What does a successful compile fail to establish?**
7. **What is a directed dependency cycle and why reject it here?**
8. **How do you verify that a test actually detects a relevant defect?**
9. **Why record build type with a timing result?**
10. **Why cannot a raw C++ struct be assumed to be a portable network format?**
11. **What is the difference between mean and tail latency?**
12. **What does an input bound need beyond a maximum list length?**

Working record

For each lab record: prediction; contract; chosen failure model; implementation revision; commands; raw results; explanation; remaining uncertainty; and a new transfer case. Use the templates directory. Separate synthetic model time from host timing, ordinary exception injection from power loss, and successful reference checks from independent student work.

Suggested completion: reading and lab evidence, five lesson questions at 9/10 or better including at least 5/6 on original questions, and a solved transfer task. Module projects use the 80/100 rubric in the capstone specification. Review blocked capabilities honestly; do not record an unavailable diagnostic as passed.

Module 16: Processor architecture and assembly

Original prerequisites: Modules 11-15.

Lab 16.1: Instruction sets, registers and calling conventions

Original practical brief: Annotate a ten-to-twenty-instruction routine with register meanings and memory accesses. Pass: predict its output for two inputs and identify caller/callee responsibilities from the chosen ABI reference.

Expanded interface and evidence contract: Native sum_u32(span<const uint32_t>) returns a modulo-2^32 sum; empty input returns zero and input is unchanged. The span must refer to valid live objects. No RISC-V execution is claimed by this helper.

Implementation and evidence steps

1. Build the native examples and run demo_16_1. Save the exact command and compiler version.
2. Make a table with columns instruction, register read, register written, memory address, and purpose. Annotate the fifteen-instruction RISC-V routine.
3. Trace the ordinary, zero-length and wraparound cases by hand. Mark which preconditions come from the caller.
4. Implement native/student/lab_16_1.hpp, run its contract checks, and compare a host-generated listing with the teaching listing.

Code check: `python3 check_lab.py 16.1 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

16.P01. Write initialization, preservation and termination statements for the assembly accumulator.

16.P02. Remove the redundant ra save/restore from this leaf routine. Which obligations remain?

16.P03. What additional evidence would support a claim that the RISC-V routine really executes correctly?

Lab 16.2: Pipelines, dependencies and branch prediction

Original practical brief: Compare dependent and independent arithmetic loops while preventing dead-code elimination. Pass: preserve observable results, inspect generated code and explain measured differences as evidence rather than universal CPU facts.

Expanded interface and evidence contract: Native sum_split(span<const uint64_t>) returns the modulo-2^64 sum without modifying input. All tail lengths are supported. Performance is an experiment, not a required speedup.

Implementation and evidence steps

1. Draw a dependency graph for one and four accumulators. Separate latency and throughput assumptions.
2. Implement sum_split and check all lengths from zero through at least two blocks plus one.
3. Run the benchmark in Debug and Release configurations, keeping the outputs separate.
4. Inspect host assembly or optimization remarks. Report the measured result and at least one alternative explanation.

Code check: `python3 check_lab.py 16.2 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

16.P04. Estimate cycles for 1000 instructions at CPI 1.5 and a 3 GHz clock under a simple average model.

16.P05. A branch-miss penalty is assumed to be 12 cycles and a trace has 25 misses. What extra term enters the toy model?

16.P06. Explain a floating-point case where regrouping can change a sum.

Lab 16.3: Caches, locality and memory hierarchy

Original practical brief: Benchmark sequential, strided and randomized access over multiple array sizes. Pass: record assumed versus measured hardware information and explain spatial locality without claiming exact cache behavior from one timing curve.

Expanded interface and evidence contract: DirectCache(lines,line_bytes): lines in 1..65536, line_bytes in 1..1048576. access(uint64 address) returns a model hit and updates the selected line; reset invalidates all lines. Every modeled access fits in one line.

Implementation and evidence steps

1. Implement DirectCache, including valid bits, configuration validation and reset.
2. Trace the six-address example and a new conflict pattern without running code.
3. Run locality_benchmark for all supplied working-set sizes in an optimized build.
4. Record equal useful element counts, index-array overhead and hypotheses that would need counters or another experiment to distinguish.

Code check: `python3 check_lab.py 16.3 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

16.P07. Trace addresses [0,16,0,16] in a one-line, sixteen-byte model.

16.P08. A loop uses 1,048,576 four-byte values and one eight-byte index per value. What active-array storage is represented?

Lab 16.4: Virtual memory, SIMD and architectural limits

Original practical brief: Inspect compiler vectorization reports for a simple loop and compare aligned/misaligned or tail cases safely. Pass: test lengths 0, 1, lane_width-1 and lane_width+1 and explain translation versus caching.

Expanded interface and evidence contract: Native add_blocks(a,b,out) requires equal lengths and valid live spans; uint32_t addition wraps. Disjoint output or exact full alias is supported; partial overlap is outside the contract. Mismatched lengths throw before modification.

Implementation and evidence steps

1. Implement add_blocks with size checks before writing any element.
2. Test lengths 0,1,3,4,5 and a longer nonmultiple; surround output slices with sentinels.
3. Generate GCC vectorization information or Clang optimization remarks using the commands in SYSTEMS_REFERENCE.
4. Explain a TLB miss, a data-cache miss and a protection fault as different events.

Code check: `python3 check_lab.py 16.4 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

16.P09. For a 4096-byte page, split address 8195 into page number and offset.

16.P10. Design a test that distinguishes an omitted tail from a correct width-four loop.

Integration project

Original brief: Trace a compiled C++ loop through a small assembly example and a cache model. Deliver an instruction/data-flow explanation and a controlled locality experiment.

Choose a bounded entity-data loop that produces a checked result. Explain its host assembly and compare it with the selected RISC-V teaching routine without treating the two targets as identical.

Required evidence

1. A register/memory trace for two inputs and the zero-length case; exact target and ABI.
2. A cache-model worksheet with stated line/entry assumptions and at least one conflict trace.
3. An equal-useful-work sequential/strided/random locality experiment at three working-set sizes.
4. A vectorization report and tail tests; explain whether observed assembly actually uses SIMD.
5. A report separating instruction semantics, analytical cost models and measured timing.

Live change: Change the line size in the model, then change the array length to one beyond a vector block. Predict the changed evidence before rerunning.

Five additional edge/failure cases

Choose five cases not copied unchanged from the demonstration. For each, state the violated assumption, expected result and test/evidence path. Include a boundary, invalid input, lifecycle transition, failure and changed workload where applicable.

Module review

1. **Explain why a correct C++ result, a correct calling convention and a fast implementation are three different claims.**
2. **A cache model predicts eight misses while a timing experiment slows much more than expected. What should happen next?**
3. **A width-four loop reads four values on every iteration until i<n. What bug appears at n=5?**
4. **What metadata must accompany an assembly or vectorization report?**
5. **Design an equal-useful-work locality comparison.**

Module 17: Operating systems and resource management

Original prerequisites: Module 16.

Lab 17.1: Processes, threads, system calls and protection

Original practical brief: Write a program that reports its process identity and runs independent worker tasks. Pass: explain what memory is shared in the chosen approach and how worker failure is reported without privileged operations.

Expanded interface and evidence contract: Python run_child(list[int],timeout=3.0) accepts at most 256 integers in 0..1000 and timeout in (0,30]. It runs only a fixed child program, returns integer pid/sum fields, and preserves the input. Errors propagate; no shell is invoked.

Implementation and evidence steps

1. Run the process demo and record that the result is correct and the process identities differ.
2. Implement run_child with the fixed child source, input validation, a timeout and checked exit status.
3. Write a resource table for parent input, encoded bytes, child list and decoded result.
4. Explain how an abnormal exit would reach the caller without converting it into an ordinary result.

Code check: `python3 check_lab.py 17.1 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

17.P01. Compare one shared-memory thread with one serialized subprocess for a mesh-analysis tool.

17.P02. Which observations should be recorded for a reproducible worker failure?

17.P03. Does setting a three-second subprocess timeout guarantee completion within exactly three seconds?

Lab 17.2: Allocation, paging and memory-mapped data

Original practical brief: Compare sequential file reads with a read-only mapped-file approach on a small disposable file. Pass: verify identical content and report virtual/resident concepts separately without promising one method is faster.

Expanded interface and evidence contract: Python file_digests(path) returns two SHA-256 hex digests for a regular file no larger than 1 MiB. The file resides in a trusted scratch directory and is not modified concurrently. Empty files are supported; no view escapes the mapping context.

Implementation and evidence steps

1. Implement streamed and read-only mapped digest paths with a 1 MiB input cap.
2. Test empty, binary, missing, directory and over-limit inputs; verify the file is unchanged.
3. Draw the file/mapping/view lifetime nesting. State the no-concurrent-writer assumption.
4. For a timing extension, define phase boundaries and trial order before collecting data.

Code check: `python3 check_lab.py 17.2 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

17.P04. A mapping begins on a page boundary and covers 12,289 bytes with 4096-byte pages. How many pages does it touch?

17.P05. Design an experiment that separates mapping setup from a complete scan.

17.P06. Why can a copied bytes(mapping) undermine an intended zero-copy experiment?

Lab 17.3: Scheduling, responsiveness and real-time constraints

Original practical brief: Simulate a scheduler or instrument a bounded producer/consumer workload without changing privileged priorities. Pass: show one contention-induced latency spike and propose a structural fix rather than only raising priority.

Expanded interface and evidence contract: Python schedule_fcfs accepts at most 128 jobs (release,duration,absolute_deadline), with bounded nonnegative integer times, positive duration and deadline >= release. It is nonpreemptive, ties use input order, and outputs include start, finish and nonnegative lateness.

Implementation and evidence steps

1. Implement the validated FCFS model and reproduce the two-job trace.
2. Construct one idle interval, one equal-release tie and one missed deadline.
3. Sketch a separate high/medium/low priority-inversion scenario with a named resource.
4. Propose a bounded handoff repair and list the ownership guarantees it needs.

Code check: `python3 check_lab.py 17.3 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

17.P07. Simulate jobs [(0,3,5),(1,2,4),(8,1,10)].

17.P08. Give one safety property and one conditional liveness property for an audio-data handoff.

Lab 17.4: Filesystems, persistence and asynchronous I/O

Original practical brief: Build a save pipeline with injected failures before, during and after serialization. Pass: recover a valid prior or new save under your documented model and never reuse an asynchronous buffer before completion.

Expanded interface and evidence contract: Python write_save/read_save use exactly version=1 and integer score in 0..1000000. write_save supports four named injection points. Readback is capped at 1024 bytes. Guarantees cover ordinary process exceptions on a trusted single-writer scratch path, not power-loss or adversarial-filesystem safety.

Implementation and evidence steps

1. Implement schema validation, bounded readback and private-temp replacement.
2. Inject before-write, partial-write, before-replace and after-replace failures.
3. Record whether the old or new state is visible at every point and verify temporary cleanup.
4. Study the asynchronous snapshot helper and explain who owns the data until future completion.

Code check: `python3 check_lab.py 17.4 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

17.P09. Create a failure table with pre-serialization, partial temporary write, before replacement and after replacement.

17.P10. Why does fsync on a temporary file alone not justify a universal durable-save claim?

Integration project

Original brief: Create a systems lab report covering processes, memory, scheduling and file I/O. Use unprivileged programs and disposable data; do not alter kernel or device settings on a primary machine.

Use only ordinary user privileges and disposable files. Compare execution and storage mechanisms while keeping the contents and useful work constant.

Required evidence

1. A process/thread ownership table, a child-process run and a reported worker error.
2. Identical stream/mapping digests, the empty-file case, bounds and map lifetime notes.
3. A scheduler trace with an explicit missed deadline; label all synthetic times as model units.
4. Before/during/after replacement save-failure tests, including observed old/new destination state.
5. An asynchronous offload lifetime diagram and completion/error check; no power-loss or hard-real-time claim.

Live change: An asynchronous save returns an exception after replacement. Explain why retry/recovery must inspect state rather than assuming that no write occurred.

Five additional edge/failure cases

Choose five cases not copied unchanged from the demonstration. For each, state the violated assumption, expected result and test/evidence path. Include a boundary, invalid input, lifecycle transition, failure and changed workload where applicable.

Module review

1. **Distinguish a process boundary, a mutex and a future.**
2. **Why can an asynchronous save still cause a visible stall?**
3. **What failure model does the reference save actually test?**
4. **How can low mean CPU usage coexist with missed audio deadlines?**
5. **A mapping digest disagrees with a streaming digest. Give a disciplined investigation order.**

Module 18: Concurrency and parallel programming

Original prerequisites: Modules 12 and 16-17.

Lab 18.1: Races, mutexes and happens-before

Original practical brief: Implement a shared counter and a compound inventory transaction with proper synchronization. Pass: stress test, inspect with an available race detector and explain why detector silence alone is not proof.

Expanded interface and evidence contract: Native Inventory(gold,stock) rejects negative initial values. buy(price) requires price>0; it changes both fields atomically under its mutex or changes neither. snapshot() returns a coherent pair. Destruction occurs only after users have joined.

Implementation and evidence steps

1. Implement Inventory with a single shared-invariant mutex and a coherent snapshot.
2. Draw the two-buyer interleaving that would break an unprotected check/debit sequence.
3. Run boundary tests and a multithreaded contention test; verify counts after joining.
4. Attempt a supported race-detector build separately from address diagnostics and record its exact outcome.

Code check: `python3 check_lab.py 18.1 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

18.P01. From gold=17 and stock=9, how many price-4 purchases can succeed?

18.P02. Give an invariant a test can check after every recorded completed transaction.

18.P03. Why is a sleeps-based ordering test a weak proof of synchronization?

Lab 18.2: Condition variables, deadlocks and cancellation

Original practical brief: Create a producer/consumer queue with graceful shutdown and failure propagation. Pass: test stopping with an empty queue, a full queue and active consumers; no thread remains blocked indefinitely.

Expanded interface and evidence contract: Native BoundedQueue(capacity) accepts 1..65536. push waits for space or closure and returns false after closure. pop waits for data or closure and returns optional<int>. close(true) drains; close(false) discards pending entries. Already popped work is not cancelled, and the owner must join users before destruction.

Implementation and evidence steps

1. Implement fixed storage, predicates, push/pop and drain/discard closure.
2. Test wraparound far beyond capacity and preserve FIFO order for one producer/consumer.
3. Test closure when empty, full and while producers or consumers are active.
4. Document which owner stops submission and joins each thread, and explain the difference between discard and rollback.

Code check: `python3 check_lab.py 18.2 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

18.P04. List a valid shutdown order for one producer and two consumers.

18.P05. Why can a notification arrive without making a consumer eligible to pop?

18.P06. Construct a deadlock involving a task that enqueues work into its own full bounded pool and then waits for it.

Lab 18.3: Atomics, memory ordering and false sharing

Original practical brief: Write a publication litmus test with a documented happens-before diagram. Measure a padded versus unpadded counter layout. Pass: do not claim stress tests prove memory-model correctness or assume a fixed cache-line size without labeling it.

Expanded interface and evidence contract: Native publish_once(uint64_t) creates one publication, returns the published value and joins the producer before local storage dies. The exercise requires a correct acquire/release or stronger protocol; stress-test success alone is insufficient.

Implementation and evidence steps

1. Implement one-shot publication without making payload atomic or using volatile as synchronization.
2. Draw the happens-before path and list the single-publication/lifetime assumptions.
3. Run repeated tests and a supported race detector, without describing the tests as a memory-model proof.
4. Run false_sharing in a Release build, preserving all measurements and recording the chosen alignment and startup costs.

Code check: `python3 check_lab.py 18.3 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

18.P07. Identify the point at which the consumer may first read payload in the one-shot protocol.

18.P08. What extra protocol is needed before freeing a node reached through an atomic pointer?

Lab 18.4: Task graphs, work stealing and scaling limits

Original practical brief: Implement a small task graph for independent entity updates followed by aggregation. Pass: compare 1, 2 and available worker counts, verify results and explain a case where more workers hurt.

Expanded interface and evidence contract: Native critical_path(durations,deps) accepts at most 1024 nodes, rejects inconsistent dimensions, invalid/duplicate edges and cycles, and returns the longest modeled path with checked uint64 arithmetic. execute_graph is a separate worked integration reference, not the function supplied by the student exercise.

Implementation and evidence steps

1. Implement critical_path and reject malformed/cyclic/overflowing models.
2. Study execute_graph, drawing the result-publication and task-claim synchronization.
3. Add an injected task failure; verify its descendants are blocked while an independent task still completes.
4. Run graph_benchmark with one, two and four workers and include thread creation and final aggregation in the stated timing boundary.

Code check: `python3 check_lab.py 18.4 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

18.P09. Compute the span of A=3, B=2 after A, C=7 independent, and D=1 after B and C.

18.P10. How should a graph handle a failed asset-decode task whose upload depends on its output?

Integration project

Original brief: Implement a bounded job system with dependency handling, shutdown, failure propagation and stress tests. Compare one-thread and multi-thread execution on a real workload.

Integrate the native queue and finite task graph. Use independent output slots and explicit dependencies rather than one giant lock around useful computation.

Required evidence

1. Queue contracts for capacity, rejection, drain, discard and destruction after joined users.
2. A happens-before explanation for the compound inventory operation and one-shot publication.
3. A graph with independent jobs, an aggregation dependency and a deliberately failed predecessor.
4. Stress and shutdown checks with external timeouts; memory/concurrency diagnostics where supported.
5. An equal-work comparison using 1, 2 and a declared available worker count; keep startup/scheduling costs explicit.

Live change: A prerequisite fails while independent work remains ready. Demonstrate that dependent work is blocked, independent work finishes, and shutdown does not wait forever.

Five additional edge/failure cases

Choose five cases not copied unchanged from the demonstration. For each, state the violated assumption, expected result and test/evidence path. Include a boundary, invalid input, lifecycle transition, failure and changed workload where applicable.

Module review

1. **State a queue safety invariant and a conditional liveness claim.**
2. **Why does join while holding a worker-needed lock risk deadlock?**
3. **Explain one-shot acquire/release publication without relying on x86 behavior.**
4. **Compute the ideal speedup for 10% serial work and four workers.**
5. **A failed graph node has descendants and an independent sibling. What does the supplied executor do?**

Module 19: Networks, storage and data services

Original prerequisites: Modules 15 and 17-18.

Lab 19.1: Network layers, latency and transport semantics

Original practical brief: Create a localhost client/server exchange with a length-prefixed message. Pass: split one message across several writes and combine several messages in one write; the receiver reconstructs them correctly.

Expanded interface and evidence contract: Frames contain a 4-byte network-order length and 1..4096 payload bytes. FrameDecoder.feed accepts bytes chunks <=8192 and emits complete bodies; finish rejects a partial tail. recv_frame reads one frame by an absolute monotonic deadline. Its socket timeout is adjusted as part of the operation.

Implementation and evidence steps

1. Implement encode_frame, incremental feed/finish and absolute-deadline recv_frame.
2. Test every possible split of two joined messages, plus one-byte fragmentation.
3. Reject zero, oversized, truncated and over-budget chunk inputs; verify decoder failure state.
4. Run the loopback integration and explain why send-call boundaries are not the test oracle.

Code check: `python3 check_lab.py 19.1 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

19.P01. A payload has 512 bytes. What is the frame size, excluding lower-layer headers?

19.P02. Give one parser test and one transport integration test that cover different risks.

19.P03. Why is TCP reliability not equivalent to authenticated player intent?

Lab 19.2: Protocol design, serialization and timeouts

Original practical brief: Add versioned framing, maximum sizes, timeout handling and request IDs to the lab protocol. Pass: reject oversized and truncated messages before dangerous allocation and test duplicate delivery.

Expanded interface and evidence contract: Python parse_request(bytes) accepts only the exact bounded version-1 schema. canonical_request validates again and emits deterministic sorted compact JSON. IDs do not authorize actions; the local gateway intentionally supplies neither public authentication nor encryption.

Implementation and evidence steps

1. Implement strict parsing and canonical_request without evaluating text.
2. Test wrong types, duplicate fields, unsupported versions, malformed text and every numeric/size boundary.
3. Use a single absolute deadline across header and body reads.
4. Specify same-ID/same-payload replay, conflicting reuse and history-full behavior before implementing retries.

Code check: `python3 check_lab.py 19.2 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

19.P04. Give three limits that must be checked before launching the native worker.

19.P05. Describe a retry policy with bounded amplification.

19.P06. Why compare stored canonical work rather than only accepting any matching ID?

Lab 19.3: Relational modeling, SQL and indexes

Original practical brief: Create a small schema with primary, foreign and uniqueness constraints. Use parameterized queries and inspect one indexed query plan. Pass: demonstrate rejected invalid references and compare a query before/after indexing.

Expanded interface and evidence contract: Python connect_store creates players, inventory and operations tables and enables foreign keys. seed_inventory atomically creates A/B and ore quantities 20/3. Explicit SQL controls transactions. A connection belongs to its thread and must be closed by its owner.

Implementation and evidence steps

1. Implement connect_store and atomic seed_inventory with explicit constraints and parameterized values.
2. Demonstrate foreign-key, uniqueness and quantity rejection through direct SQL operations.
3. Use the query-plan helper to compare an item lookup before and after an item_id index, preserving returned rows.
4. Complete the PostgreSQL comparison worksheet, labeling any SQL companion that was not executed on a PostgreSQL server.

Code check: `python3 check_lab.py 19.3 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

19.P07. Which query is naturally served by the leading player_id field of the composite key?

19.P08. Does a successful transaction context automatically close a Python sqlite3 connection?

Lab 19.4: Transactions, consistency and distributed failure

Original practical brief: Implement an inventory transfer with transactions and forced failure between updates. Pass: verify conservation, rollback and duplicate-request handling; document what is not guaranteed across processes or services.

Expanded interface and evidence contract: Python transfer requires distinct known players, valid identifiers and quantity 1..1000000. It conserves quantity, enforces destination capacity, persists at most 1024 replay records, and rejects conflicting ID reuse. ResponseLost after_commit represents committed work with a lost reply, not rollback.

Implementation and evidence steps

1. Implement transfer with exact input validation and one transaction for both quantities and the replay record.
2. Inject after_debit, before_commit and after_commit and check all relevant tables.
3. Test duplicate and conflicting IDs, unknown players, insufficient source quantity and history-full rejection.
4. Explain which guarantees hold inside the one database and which are not supplied across services or after history deletion.

Code check: `python3 check_lab.py 19.4 --student`. The worked solution can be checked separately with `--reference`.

Additional problems

19.P09. Give an assertion that distinguishes a rollback from an incorrectly half-applied transfer.

19.P10. What happens to an old retry if its operation record was deleted?

Integration project

Original brief: Build a local-only client/server message exercise and a small relational game-data service. Test malformed framing, transaction rollback and duplicate requests without exposing a public service.

Build a loopback or in-process protocol harness and a relational inventory operation. State exactly which behavior belongs to transport, parsing and persistence.

Required evidence

1. A four-byte big-endian framing specification, fragmented/coalesced parser checks and a truncated-EOF test.
2. An absolute receive deadline, byte/field bounds and strict versioned schema validation.
3. Primary/foreign/unique/check constraints, parameterized queries and a before/after index plan.
4. Atomic transfer with injected failure, conserved quantity and an operation replay record in the same transaction.
5. A lost-response retry and conflicting-ID case; retain history and state the replay window.

Live change: The first reply is lost after commit, then the same ID arrives with a different payload. Show that the old effect remains and the conflicting request is rejected.

Five additional edge/failure cases

Choose five cases not copied unchanged from the demonstration. For each, state the violated assumption, expected result and test/evidence path. Include a boundary, invalid input, lifecycle transition, failure and changed workload where applicable.

Module review

1. **Why can a receive timeout leave the remote operation's outcome unknown?**
2. **Which three records must be committed together for a deduplicated transfer?**
3. **Does a foreign-key definition alone establish enforcement in the SQLite exercise?**
4. **What happens when the same request ID is reused with a different payload?**
5. **What is excluded by the loopback gateway's deployment scope?**

Module 20: Engineering practice, security and maintainability

Original prerequisites: Volumes 01-04.

Lab 20.1: Requirements, architecture and design records

Original practical brief: Write six measurable requirements and two decision records for the simulation. Pass: every requirement has an acceptance test and each decision includes a rejected alternative and a cost.

Expanded interface and evidence contract: Evidence lab: requirements, traceability and two architecture decision records. The demonstration only detects missing evidence entries; it does not certify architecture quality or execute named tests.

Implementation and evidence steps

1. Write six measurable requirements for your implementation and connect each to an acceptance check.
2. Create separate resource/ownership and request-lifecycle views; identify the commitment point.
3. Write two decision records with at least one rejected alternative and a cost each.
4. Review a changed constraint, such as capacity two or no subprocess creation, and state which decision must be revisited.

Review check: preserve the documents, observed test results and reviewer findings; there is no automatic lesson pass from the demonstration script.

Additional problems

20.P01. Rewrite "the server must never crash" as a bounded, testable requirement.

20.P02. Name one cost of using a subprocess for the tiny arithmetic operation.

20.P03. What should change in an ADR when a result contradicts the original assumption?

Lab 20.2: Defensive programming and threat modeling

Original practical brief: Threat-model the save parser and local service. Add bounds checks and a small malformed-input fuzz harness. Pass: cap work/memory, preserve useful errors and record residual risks instead of claiming complete security.

Expanded interface and evidence contract: Evidence lab: threat model plus bounded fuzz/mutation report against this package only. No network scanning, public deployment or untrusted executable execution is required. Claims must distinguish tests, design arguments and untested assumptions.

Implementation and evidence steps

1. Build a threat table listing assets, entry points, trust assumptions, failure effects and controls.
2. Add a bounded malformed-input harness plus valid-message mutations; retain seeds and cases.
3. Check that saturation, invalid work and worker failure do not create committed result rows.
4. Record residual risks: no local-user authentication, no hostile-native-code sandbox, no power-loss proof, and dependence on private writable storage.

Review check: preserve the documents, observed test results and reviewer findings; there is no automatic lesson pass from the demonstration script.

Additional problems

20.P04. Write a threat-model row for untrusted length headers.

20.P05. Write a residual-risk statement for the native worker.

20.P06. Why avoid calling a random parser test a penetration test of the operating system?

Lab 20.3: Build automation, dependencies and release discipline

Original practical brief: Create a local clean-build script that runs tests and produces a versioned archive plus manifest. Pass: run it from a fresh build directory and deliberately break a test to confirm the pipeline stops.

Expanded interface and evidence contract: Evidence lab: clean build/test log, failed-pipeline demonstration, dependency inventory, versioned source release and extraction/rebuild evidence. A hash manifest is an integrity aid, not an authenticity certificate.

Implementation and evidence steps

1. Run the supplied verification flow from a fresh build directory and preserve its output.
2. Run the explicit pipeline failure injection and confirm a new release is not produced.
3. Fill a dependency inventory and generate a manifest containing source hashes and tool versions.
4. Extract a successful source release into a clean folder and rebuild rather than shipping a Linux binary as a universal executable.

Review check: preserve the documents, observed test results and reviewer findings; there is no automatic lesson pass from the demonstration script.

Additional problems

20.P07. List five facts needed to identify the origin of a tested native artifact.

20.P08. Explain why adding a checksum file to a download does not automatically authenticate it.

Lab 20.4: Ethics, accessibility, teamwork and sustainable delivery

Original practical brief: Run an accessibility review and a blameless failure retrospective on the project. Pass: identify at least three actionable changes, assign acceptance evidence and document third-party attribution honestly.

Expanded interface and evidence contract: Evidence lab: usability/accessibility review, blameless incident report, attribution/dependency record and Gate D handoff. Do not mark human review complete because an automated example printed a proposed message.

Implementation and evidence steps

1. Have another person follow the README without verbal help and record observed problems.
2. Review text-only status, keyboard operation, error recovery and unnecessary data exposure.
3. Write a retrospective for one injected failure with three actionable improvements and acceptance evidence.
4. Complete Gate D with an attribution record, known limitations and a live independent change.

Review check: preserve the documents, observed test results and reviewer findings; there is no automatic lesson pass from the demonstration script.

Additional problems

20.P09. Write three actionable improvements after a cancellation incident.

20.P10. What must an honest portfolio say about using the reference service?

Integration project

Original brief: Complete Gate D: a reviewed systems application with a design record, threat model, CI-style local test pipeline, dependency inventory and an incident/recovery exercise.

Assemble the earlier investigations into the local workload-processing application. Complete the Gate D specification rather than adding a public deployment.

Required evidence

1. Six measurable requirements, two reviewed decision records and test-to-requirement traceability.
2. A thread/resource ownership map, protocol/schema and explicit commit/cancel boundary.
3. A bounded threat model and malformed-input harness; residual risks and diagnostic limits.
4. A fresh build, versioned source archive and manifest; intentionally failed tests prevent release.
5. An operator/accessibility review, blameless recovery retrospective, dependency inventory and independent handoff.

Live change: Give a reviewer a fresh extraction. Reproduce one recovery case, explain one measured bottleneck and implement a small changed requirement without copying a solution.

Five additional edge/failure cases

Choose five cases not copied unchanged from the demonstration. For each, state the violated assumption, expected result and test/evidence path. Include a boundary, invalid input, lifecycle transition, failure and changed workload where applicable.

Module review

1. **What distinguishes the original curriculum requirements from the new ForgeQueue design?**
2. **Why must pipeline failure injection precede trusting a green release script?**
3. **Give one claim that code tests cannot fully establish.**
4. **Why is rejecting new unique jobs when history is full sometimes the correct behavior?**
5. **What must a Gate D handoff let a reviewer reproduce?**

Gate D submission checkpoint

Use `CAPSTONE_SPEC.pdf` for the exact ForgeQueue realization, bounds, failure matrix and scoring. Submit at least twenty learner-authored acceptance tests plus the native concurrency evidence and the systems investigation. The supplied reference suite is a baseline, not a substitute for independent requirements and reasoning.